5.2 指令中的地址码

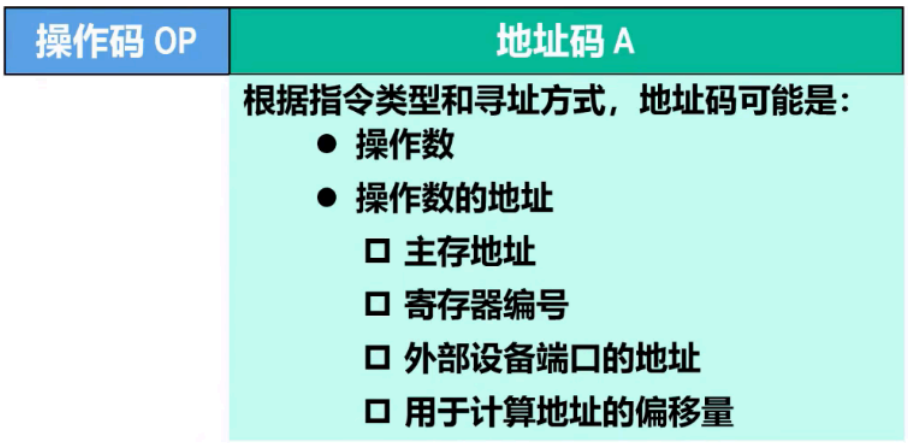
考纲定位：计算机组成原理 · 第 5 章 指令系统（5.2） | 重要度 ★★★★ | 常见考法：选择题（地址码个数、二地址三种形式、一地址隐含操作数、零地址、指令字长计算）

一句话核心：地址码个数决定“操作数从哪来、结果到哪去”；地址码越少，指令越短，隐含约定越多

一、地址码的内容与分类

1.1 内容

指令的一般格式



提炼

- 地址码 A 可能是操作数本身、主存地址、寄存器编号、I/O 端口地址、偏移量
- 具体含义由指令类型和寻址方式决定，不能只看字段名字

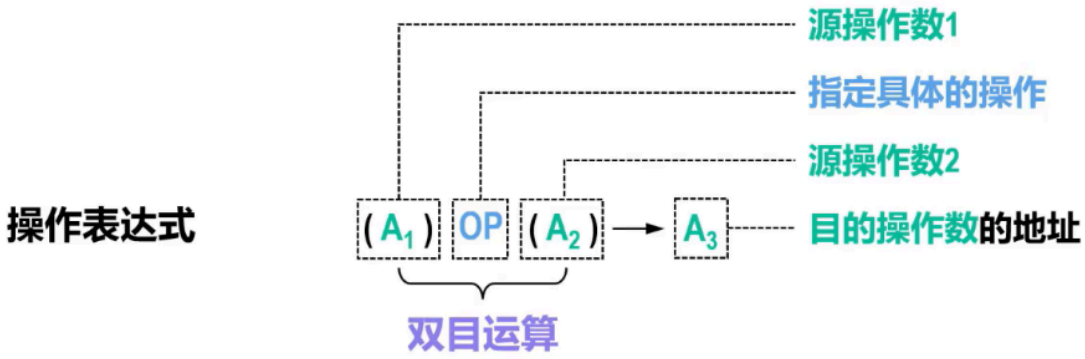
易错

- 地址码字段的位数 ≠ 主存地址位数；寄存器编号只需 $\lceil \log_2(\text{寄存器数}) \rceil$ 位
- 立即寻址的地址码字段装的是操作数本身，不是地址

1.2 分类

1. **三地址指令**

操作码 OP	地址码 A_1	地址码 A_2	地址码 A_3
	源操作数1的地址	源操作数2的地址	目的操作数的地址



举例

【举例】



指令字长为$8+8+8+8=32$位

操作数的寻址范围为$2^8=256$

- **如果操作数的寻址范围更大，则需要地址码的位数就会更大，指令就会更长。**
- **因此，三地址指令的地址码字段中的3个地址码、全部用于存储单元地址的情况并不多见，较为常见的是全部为寄存器（例如MIPS的指令），这样可以减少指令长度。**

提炼

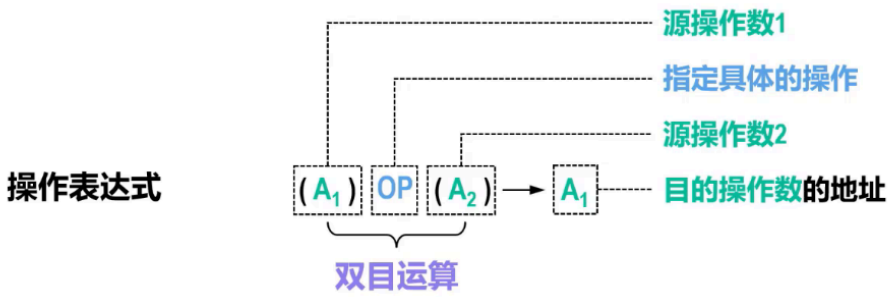
- 格式 OP A1 A2 A3，语义 A1 op A2 → A3
- 优点：源操作数不被破坏，程序表达清晰
- 缺点：指令字长最长；若三个地址都指向主存，取指、取两个源操作数、写结果，访存次数多
- MIPS 的三地址指令通常三个地址都是寄存器编号，以缩短指令字长

易错

- 三地址指令的“三”指地址码个数，不是操作数个数；立即数指令也可以有三地址格式
- 三地址指令不一定比二地址慢；若三个都是寄存器操作数，取操作数的访存次数为 0

2. **二地址指令**

操作码 OP	地址码 A_1	地址码 A_2
	源操作数1的地址 目的操作数的地址	源操作数2的地址



- 与三地址指令相同，二地址指令同样是为双目运算类设计的。
- 但是，二地址指令为了压缩指令字长，将运算结果直接存放到源操作数1的地址A_1中。

按照两操作数的存放位置不同，分三种情况

操作码 OP	地址码 A_1	地址码 A_2
	源操作数1的地址 目的操作数的地址	源操作数2的地址

- 与三地址指令相同，二地址指令同样是为双目运算类设计的。
- 但是，二地址指令为了压缩指令字长，将运算结果直接存放到源操作数1的地址A_1中。



寄存器　寄存器

存储器　寄存器

存储器　存储器

- Intel x86系列计算机主要采用RR（寄存器-寄存器）型指令和RS（寄存器-存储器）型指令。
- MIPS计算机主要采用RR（寄存器-寄存器）型指令。

提炼

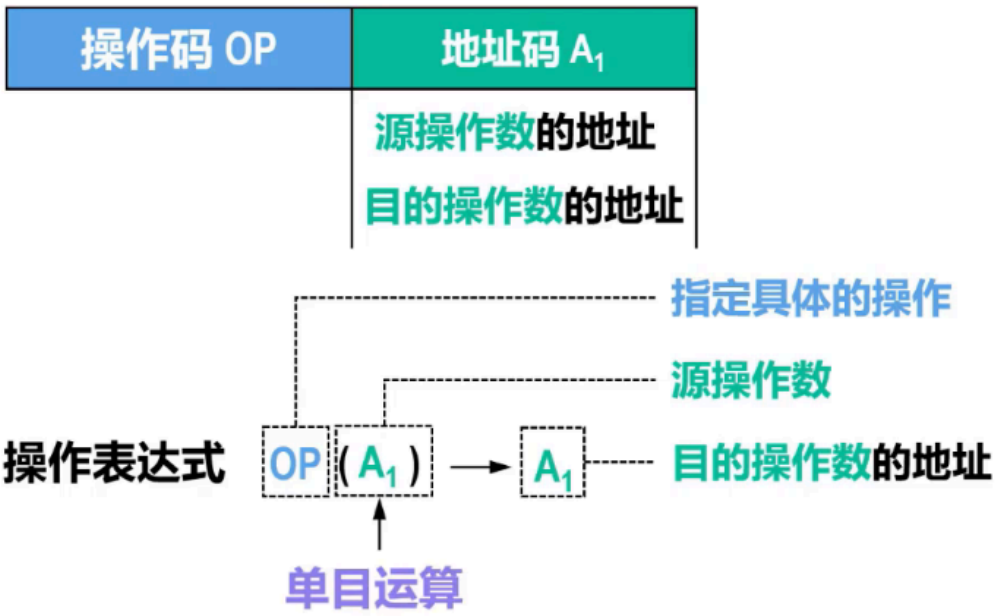
- 格式 OP A1 A2，默认 A1 既是源操作数 1 的地址，又是目的地址：A1 op A2 → A1
- 三种组合
  - RR（寄存器-寄存器）：速度最快，指令最短
  - RS（寄存器-存储器）：x86 常用，一个操作数在寄存器，一个在主存
  - SS（存储器-存储器）：速度最慢，访存最多
- 用“结果覆盖源操作数 1”压缩指令字长，代价是源操作数 1 被破坏

易错

- 二地址指令不是“两个操作数都在主存”；RR/RS/SS 是操作数存放位置，不是寻址方式
- “结果覆盖 A1”是默认约定；若题目要求保留 A1，需要额外指令或改用三地址格式
- x86 的 RR/RS 与 MIPS 的 RR 不要混为一谈

3. **一地址指令**

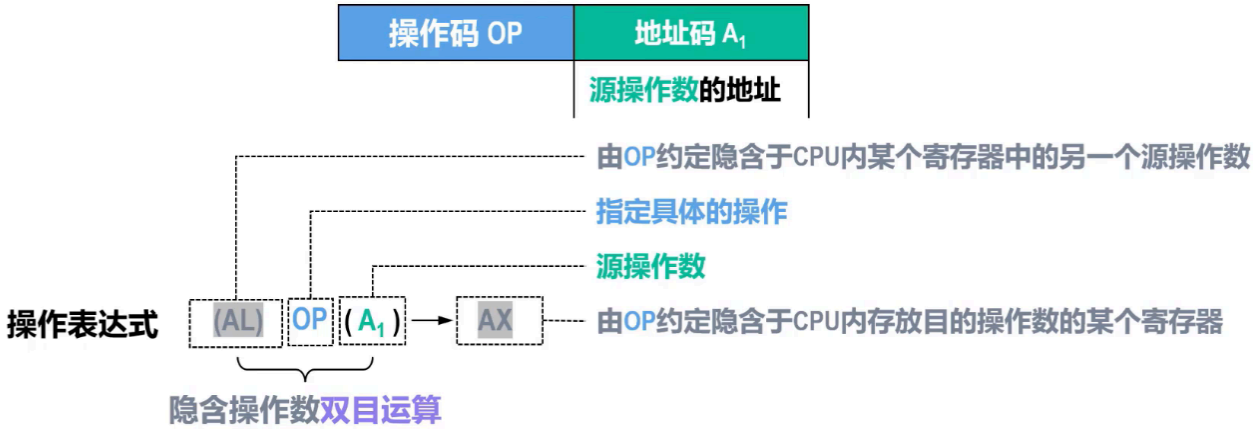
情况一：地址为源操作数和目的操作数的地址



提炼

- 格式 OP A1，语义 A1 op (隐含操作数) → A1
- 单目运算常用：取反、加 1、减 1、移位

情况二：地址只为源操作数的地址



- **例如x86架构中 Mul BL 指令，会将AL寄存器中的数据与BL寄存器中的数据相乘，结果存放在AX寄存器中。**
- **隐含操作数双目运算，进一步缩短了双目运算类指令的长度。**

提炼

- 格式 OP A1，另一个源操作数与目的地址由 OP 隐含约定
- 例：x86 的 MUL BL 约定 AL × BL → AX

易错

- 一地址指令不等于“只有一个操作数”；可能是单目运算，也可能是双目运算的隐含形式
- 隐含操作数通常放在累加器 ACC 或由指令固定约定，题目未说明时不要自行假设

4. **零地址指令**



- **指令本身不需要操作数，例如以下x86指令**
  - **空指令 NOP**
  - **等待指令 WAIT**
  - **停机指令 HLT**
  - **程序返回指令 RET**
- **指令需要一个操作数，该操作数隐含于CPU内的某个寄存器，例如以下x86指令**
  - **用于将加法运算结果调整为压缩BCD格式的指令 DAA**
  - **用于将减法运算结果调整为压缩BCD格式的指令 DAS**

提炼

- 指令本身不需要操作数：NOP、WAIT、HLT；x86 的 RET
- 或操作数隐含在 CPU 寄存器/栈顶：DAA、DAS、栈操作
- 零地址指令的“零”指地址码字段为空，不是没有操作数

易错

- x86 的 RET 是零地址指令，返回地址隐含在栈顶；MIPS 的返回指令 `jr` 使用 RA 寄存器，不是零地址
- 零地址指令不一定不访存；栈操作要读写主存

二、地址码个数与指令字长

地址码个数	典型语义	优点	缺点	典型场景
三地址	A1 op A2 → A3	源操作数不被破坏	指令最长，访存多	需要保留源值的运算
二地址	A1 op A2 → A1	指令较短，兼容性好	破坏 A1	RR/RS/SS 通用运算
一地址	(ACC) op A1 → ACC	指令更短	依赖隐含寄存器	单目运算、累加器架构
零地址	栈顶 op 次栈顶 → 栈顶	指令最短	依赖栈	栈式计算机、NOP/RET

提炼

- 指令字长固定时，地址码越多，每个地址码分到的位数越少，寻址范围越小
- 地址码个数影响取操作数的访存次数，但取指本身也要访存；计算总访存次数时不要漏取指

易错

- 地址码个数与操作数个数不是一一对应
- 二地址指令的 RR/RS/SS 影响访存次数，不影响“二地址”这个分类
- 地址码个数减少，操作码可扩展位数增加，二者是同一字长下的零和关系

知识关联：地址码与操作码共享指令字长（5.1 指令的一般格式与指令字长）；地址码中的寄存器编号与寻址方式字段（5.4 寻址方式概述）；二地址 RS/SS 的访存次数影响流水线（6.28 指令流水线——数据通路）